AWS Certified Generative AI Developer – Professional

AIP-C01 — EASY-ENGLISH ONE-STOP REVISION + EXAM GUIDE

Exam area	Weight
Foundation Model Integration, Data Management, and Compliance	31%
Implementation and Integration	26%
AI Safety, Security, and Governance	20%
Operational Efficiency and Optimization	12%
Testing, Validation, and Troubleshooting	11%

How to use this: First learn the simple meaning. Then learn the exam clue. Finally, practise the scenario.

Important: AWS service features, pricing, model availability, and exam details can change, so use the current official AWS material for final verification.

1. The AWS GenAI Application — The Big Picture

Do not try to memorize every AWS service separately. First understand how the pieces fit together.

Simple architecture:

User → Login → API → Application → Data/RAG → Bedrock/Agent → Answer

Around this flow we add security, monitoring, auditing, queues, workflows, and notifications.

Need	Service / concept	Easy meaning
User login	Cognito	Lets application users sign in.
API entry point	API Gateway	Receives requests from the application.
Run code	Lambda	Runs code without managing servers.
Store documents	S3	Stores files such as PDFs.
Generate AI response	Bedrock	Gives access to foundation models.
Company knowledge chatbot	Q Business	Answers questions using company information.
Coding assistant	Q Developer	Helps developers write and understand code.
Multi-step workflow	Step Functions	Controls a sequence of steps.
Queue work	SQS	Holds work until a worker can process it.
Send notifications	SNS	Sends/fans out notifications.
Monitor	CloudWatch	Shows logs, metrics, alarms, and dashboards.
Audit AWS actions	CloudTrail	Records API/account activity.
Permissions	IAM	Controls what AWS identities can do.
S3 sensitive-data discovery	Macie	Finds sensitive data in S3.
Compliance/configuration	AWS Config	Checks resource configuration and compliance.
Custom ML	SageMaker AI	Builds/trains/deploys ML models.

Memory line: Login → API → Code → Data → AI → Queue/Workflow → Notify → Monitor → Audit → Secure.

2. What You Should Know Before the Exam

Your original document recommends knowing the basics of AWS services and understanding the SageMaker ML pipeline.

- AWS cloud fundamentals.
- IAM basics.
- CloudWatch and CloudTrail.
- AWS Config and security services.
- Basic understanding of SageMaker and the ML pipeline.
- Basic understanding of AWS AI services.
- Scenario-based thinking rather than only definitions.

Do not go too deep into traditional ML. For this exam revision, understand where SageMaker fits and how it differs from Bedrock.

3. Core AWS Services — Easy Explanation

Amazon Cognito

- What it does: Manages users for your application.
- Use it for: sign-up, sign-in, password recovery, social login.
- Exam clue: “Users must log in before using the application.”
- Remember: Cognito = application user login.

API Gateway

- What it does: Creates and manages APIs.
- Use it for: receiving requests from web/mobile applications.
- Exam clue: “Expose an API”, “API entry point”, “rate limiting”.
- Remember: API Gateway = front door for APIs.

AWS Lambda

- What it does: Runs code without managing servers.
- Use it for: event-driven processing and application logic.
- Example: S3 upload triggers Lambda → Lambda processes the document.
- Remember: Lambda = worker.

Amazon S3

- What it does: Stores objects/files.
- Use it for: PDFs, documents, images, datasets, and application files.
- GenAI use: common source location for Knowledge Base documents.
- Remember: S3 = filing cabinet.

Amazon Bedrock

- What it does: Gives applications access to foundation models through managed APIs.
- Use it for: text generation, GenAI applications, RAG, agents, guardrails, prompt management, model evaluation, and related capabilities.
- Exam clue: “Which AWS service provides access to foundation models?”
- Remember: Bedrock = GenAI platform.

Amazon Q Business

- What it does: Enterprise AI assistant.
- Use it for: employees asking questions about company information.
- Example: “What is our leave policy?”
- Remember: Q Business = company knowledge assistant.

Amazon Q Developer

- What it does: AI assistant for developers.
- Use it for: code generation, explanation, debugging, tests, documentation.
- Exam clue: “Developer needs help writing code.”
- Remember: Q Developer = coding assistant.

4. Workflow and Messaging Services

Service	Simple meaning	Exam clue
Step Functions	Controls a multi-step workflow.	Many steps must happen in order.
SQS	Stores work in a queue.	Traffic spike / decouple / process later.
SNS	Sends notifications to subscribers.	Notify / publish / fan-out.
EventBridge	Routes events to targets.	Event-driven integration and routing.

Example: PDF upload → extract text → summarize → fraud check → save result → notify customer.

Why Step Functions? Because the application has several steps and needs orchestration.

Why SQS? If 10,000 requests arrive together, the queue can hold them until workers process them.

Why SNS? When processing finishes, publish a notification such as “Your claim summary is ready.”

Easy memory: Step Functions = Manager | SQS = Waiting line | SNS = Announcement.

5. Security and Governance — Easy Version

Need	Service	Easy meaning
AWS permissions	IAM	Controls access to AWS resources.
Application user login	Cognito	Manages users of your app.
Employee SSO	IAM Identity Center	One login for AWS accounts/apps.
Sensitive data in S3	Macie	Finds sensitive information in S3.
Model safety controls	Bedrock Guardrails	Controls unsafe inputs/outputs and policies.
AWS API audit	CloudTrail	Records who did what through AWS APIs.
Monitoring	CloudWatch	Shows logs, metrics, alarms, dashboards.
Configuration compliance	AWS Config	Checks resource configuration against rules.
Threat detection	GuardDuty	Detects suspicious/malicious activity.
Security findings	Security Hub	Centralizes security findings.
Audit evidence	Audit Manager	Helps automate collection of audit evidence.

IAM vs Cognito: IAM is mainly about AWS permissions. Cognito is mainly about application users.

CloudTrail vs CloudWatch: CloudTrail asks “Who did what?” CloudWatch asks “How is my system behaving?”

Macie vs Guardrails: Macie finds sensitive data in S3. Guardrails control GenAI interactions.

6. Amazon Bedrock — The Most Important Service

Think of Bedrock as the AWS platform you use when you want to build a Generative AI application using foundation models.

Bedrock feature	Easy meaning
Foundation Models	Ready-to-use AI models.
Knowledge Bases	Helps build RAG applications.
Agents	Lets an AI application use tools/actions to complete tasks.
Guardrails	Adds safety and policy controls.
Prompt Management	Stores and manages prompts.
Model Evaluation	Helps evaluate/compare models.
Prompt Caching	Can reduce repeated work for eligible repeated prompt content.

Exam clue: If the question asks for managed access to foundation models, think Bedrock.

Bedrock vs SageMaker: Bedrock is mainly for using foundation models and GenAI capabilities. SageMaker is the broader ML platform for building/training/deploying ML models.

7. RAG — Understand This Clearly

RAG = Retrieval-Augmented Generation.

Instead of asking the model to answer only from what it already knows, we first find useful information from our own data and give that information to the model.

Simple flow:

Documents → Split into chunks → Create embeddings → Store in vector store → User asks question → Find relevant chunks → Give chunks to model → Generate answer

Term	Easy meaning
Chunking	Break a large document into smaller pieces.
Embedding	Turn text into numbers/vectors that represent its meaning.
Vector store	Stores those vectors so we can search them.
Metadata	Extra information such as department, date, document type, or tenant.
Vector search	Find content with similar meaning.
Hybrid search	Uses both keyword and semantic search.
Reranking	Reorders retrieved results so the most useful ones come first.
Query expansion/decomposition	Changes one question into better search queries.
Freshness	Makes sure the retrieval data is not outdated.

When should you think RAG? Private company documents, changing information, large document collections, or answers that need supporting information.

RAG is not fine-tuning. RAG gives the model information at query time. Fine-tuning changes/customizes model behavior.

8. Chunking — The Simple Way to Think About It

Chunking matters because the model cannot search a huge document as one useful piece. We split the document so retrieval can find the right part.

Problem	What may happen
Chunks too small	Important context may be lost.
Chunks too large	Retrieval may bring too much irrelevant information.
Poor chunk boundaries	The answer may be split across chunks.
Poor metadata	The system may retrieve documents the user should not see.
Too many retrieved chunks	Prompt becomes large, slow, and expensive.

Exam approach: If retrieval is poor, do not immediately blame the foundation model. Check chunking, embeddings, metadata, search, and reranking.

9. Prompt Engineering — What You Need to Remember

Technique	Easy meaning
Clear instruction	Tell the model exactly what you want.
Role	Tell the model what role it should follow.
Examples / few-shot	Show examples of the expected answer.
Delimiters	Clearly separate instructions from data.
Structured output	Ask for a predictable format such as JSON when needed.
Temperature	Controls how varied/random the output can be.
Top-p / top-k	Control which token choices can be sampled.
Prompt Management	Manage reusable/versioned prompts.
Prompt Flows	Build a sequence of prompt-processing steps.
Prompt caching	Reuse eligible repeated prompt content to improve efficiency.

Simple rule: If you cannot clearly explain what the model should do, the prompt is probably not clear enough.

10. Agents — Easy Explanation

A normal chatbot mainly answers. An **agent** can decide what action it needs to take and use tools to complete the task.

Example: User asks, “Check my claim status and send me the result.”

The agent may decide: check claim system → get result → prepare response → send notification.

Concept	Easy meaning
Tool	A function/API the agent can call.
Tool calling	The model decides which tool to use and provides arguments.
Memory	Stores useful information for future interactions.
State	Tracks what has happened in the task.
Human approval	Stops and asks a person before a risky action.
MCP	A standard way for AI applications/agents to interact with tools and resources.
Multi-agent	Multiple specialized agents work together.
Parameter validation	Check tool inputs before executing the tool.
Circuit breaker	Stop repeated calls when a dependency keeps failing.

Important: Never assume model-generated tool parameters are safe. Validate them before the real action happens.

11. Amazon Bedrock AgentCore — Easy Memory Map

AgentCore part	Think of it as
Runtime	Where agents/tools run.
Gateway	A controlled gateway for agentic traffic and tools; supports MCP-compatible tool patterns.
Memory	Agent memory.
Identity	Agent identity and credentials.
Observability	See what the agent and its tools are doing.

Easy memory: Run → Gateway → Remember → Identity → Observe.

Do not reduce AgentCore to only MCP. MCP is one important tool-interaction pattern; AgentCore covers a wider agent production platform.

12. GenAI Safety — The Exam Way

Think about safety at every stage: user input → retrieval → model → tools → output.

Problem	What to think about
Prompt injection	Treat user/retrieved text as untrusted data. Keep instructions separate and validate actions.
Jailbreak	Use safety controls and testing to prevent attempts to bypass restrictions.
Hallucination	Improve grounding/retrieval and validate the answer.
PII leakage	Detect/mask sensitive information and restrict access.
Unsafe tool action	Use authorization, parameter validation, and output checks.
Sensitive company data	Use least privilege and appropriate data protection controls.

Defense in depth: Do not expect one control to solve every security problem.

13. Evaluation — How Do We Know the GenAI App Is Good?

Metric / check	Simple question
Relevance	Did it answer the right question?
Accuracy	Is the answer correct?
Groundedness	Is the answer supported by the retrieved information?
Fluency	Is the answer clear and readable?
Safety	Did it avoid unsafe content?
Bias/fairness	Does it behave unfairly for some groups?
Task success	Did the agent actually complete the job?
Latency	Was it fast enough?
Cost	Was the quality worth the cost?

Useful approaches: Bedrock Model Evaluations, golden test datasets, human evaluation, LLM-as-a-Judge, A/B testing, and regression testing.

Regression testing: When you change the model, prompt, retrieval, chunking, embeddings, or guardrails, test again. A change that improves one case can break another.

14. Cost and Performance — Easy Exam Logic

If you want…	Think about…
Lower token cost	Shorter prompts and less unnecessary context.
Lower repeated prompt cost/latency	Prompt caching when eligible.
Lower model cost	Use a smaller model for simple tasks.
Lower retrieval cost	Retrieve fewer, more relevant chunks.
Handle traffic spikes	SQS / asynchronous processing.
Better user-perceived latency	Streaming.
Faster workflow	Run independent steps in parallel when possible.
Recover from temporary failure	Bounded retries + exponential backoff.
Avoid repeated failures	Circuit breaker.

Exam trick: Do not automatically choose the biggest model. First ask what quality the use case actually needs.

15. Troubleshooting — Find the Layer That Is Broken

Problem	First things to check
Wrong answer	Retrieved data, prompt, chunking, embeddings, filters, reranking.
Hallucination	Was the correct information retrieved? Is the answer grounded?
Too much context	Reduce chunk size/top-k and remove unnecessary context.
Slow response	Model latency, retrieval, serial tool calls, lack of streaming.
High cost	Token count, model choice, repeated prompts, caching.
Throttling	Concurrency, quotas, retry/backoff, queues.
Tool fails	Arguments, schema, IAM/identity, timeout, downstream service.
Agent keeps looping	Stopping condition, state, tool result, retry limits, circuit breaker.
Unsafe response	Guardrails, input/output checks, validation.

Very useful exam rule: Fix the correct layer. CloudTrail cannot fix hallucinations. Increasing temperature cannot fix missing documents. SQS cannot fix a bad prompt.

16. Must-Know Comparisons

Service A	Service B	Key difference
Bedrock	SageMaker AI	Bedrock = foundation-model/GenAI platform; SageMaker AI = ML platform.
CloudWatch	CloudTrail	CloudWatch = monitor; CloudTrail = audit.
Macie	Guardrails	Macie = sensitive-data discovery in S3; Guardrails = GenAI safety.
Cognito	IAM Identity Center	Cognito = application users; Identity Center = workforce SSO.
IAM	Cognito	IAM = AWS permissions; Cognito = application users.
Q Business	Q Developer	Q Business = enterprise assistant; Q Developer = coding assistant.
SQS	SNS	SQS = queue; SNS = publish/notify.
Step Functions	Agent	Step Functions = defined workflow; Agent = dynamic tool decisions.
RAG	Fine-tuning	RAG = retrieve knowledge; fine-tuning = customize behavior.

17. Scenario Revision — Read the Clue, Pick the Service

Scenario clue	Think
“Users need to log in before using the chatbot.”	Cognito
“Developer needs help writing Python code.”	Q Developer
“Employees ask questions about company policies.”	Q Business
“Use foundation models through a managed AWS service.”	Bedrock
“Answers must use changing private documents.”	RAG / Knowledge Base
“10,000 requests arrive together.”	SQS / decoupling
“Run upload → extract → summarize → notify.”	Step Functions
“Send a notification when processing finishes.”	SNS
“Who deleted the resource?”	CloudTrail
“Show latency and errors.”	CloudWatch
“Check whether the S3 bucket violates a configuration rule.”	AWS Config
“Find sensitive information in S3.”	Macie
“Stop unsafe model output.”	Bedrock Guardrails
“Lambda needs permission to invoke Bedrock.”	IAM
“Agent needs to call an external tool.”	Tool calling / AgentCore / MCP pattern
“Tool receives invalid parameters.”	Validate parameters before execution

Exam habit: Circle the words that describe the requirement. The requirement usually tells you the answer.

18. How to Answer AWS Scenario Questions

Step 1: Read the question and identify the actual goal.

Step 2: Find the important constraint: cost, speed, security, scale, accuracy, privacy, automation, or simplicity.

Step 3: Identify the AWS layer: identity, API, compute, data, RAG, model, agent, safety, monitoring, or audit.

Step 4: Choose the simplest solution that satisfies the requirement.

Step 5: Remove answers that add unnecessary services or operational work.

Step 6: For multiple-response questions, check every selected answer independently.

Step 7: If two answers look correct, look for the extra requirement. That extra sentence is often the tie-breaker.

Remember: The exam is scenario-based. Knowing a definition is not enough; you must know when to use the service.

19. Final 30-Minute Revision Sheet

Bedrock: Foundation models + GenAI capabilities.

RAG: Retrieve useful data → give it to model → generate answer.

Knowledge Base: Helps implement RAG over your data.

Agent: AI that can decide and use tools to complete tasks.

Guardrails: Safety controls.

Prompt Management: Manage prompts.

Prompt caching: Reuse eligible repeated prompt content.

Lambda: Run code.

API Gateway: API front door.

Step Functions: Workflow manager.

SQS: Queue / waiting line.

SNS: Notification / fan-out.

Cognito: Application login.

IAM: AWS permissions.

Identity Center: Employee SSO.

Macie: Sensitive data in S3.

CloudWatch: Monitor.

CloudTrail: Audit.

AWS Config: Configuration/compliance.

Q Business: Enterprise assistant.

Q Developer: Coding assistant.

SageMaker AI: ML platform.

AgentCore Runtime: Run agents.

AgentCore Gateway: Gateway for agentic traffic/tools.

AgentCore Memory: Agent memory.

AgentCore Identity: Agent identity.

AgentCore Observability: See what agents/tools are doing.

One sentence to remember: “Authenticate the user, expose the API, run the logic, retrieve the right data, use the right model, protect the interaction, handle failures, monitor everything, and audit important actions.”